Airline Ticketing System API

1. Introduction

This project aims to design and implement a scalable airline ticketing backend system using modern software architecture principles. The system supports core airline operations such as flight management, ticket purchasing, and passenger check-in.

The project focuses on building a microservice-based architecture using an API Gateway to manage client requests and ensure system scalability, security, and performance.

Additionally, rate limiting is implemented to prevent excessive traffic and protect backend services. The system is deployed on AWS to simulate a real-world production environment.

2.Project Goal

This project is a backend API for an airline ticketing system. It allows authorized users to add flights, and customers to search flights, purchase tickets, perform check-in, and view passenger lists.

2.1 System Architecture

The system follows a layered architecture with an API Gateway.

Client (Postman / Swagger)

↓

API Gateway (Spring Cloud Gateway)

↓

Backend Service (Spring Boot)

↓

Database (MySQL)

All client requests pass through the API Gateway, where routing and rate limiting are handled before reaching the backend service.

3.Technologies

Backend:

- Java
- Spring Boot

Database:

- MySQL

Authentication:

- JWT

API Documentation:

- Swagger / OpenAPI (springdoc-openapi)

API Gateway:

- Spring Cloud Gateway

Load Testing:

- k6

Deployment:

- EC2

3.1 API Gateway Features

• All incoming requests are routed through the API Gateway (Spring Cloud Gateway).

• Gateway acts as a single entry point for all backend services.

• Rate limiting is implemented at the gateway level for the flight search endpoint.

3.2 Rate Limiting

Rate limiting is implemented at the API Gateway layer using a custom filter.

The endpoint /api/v1/flights/search is limited to 3 requests per day per client based on IP address. When the request limit is exceeded, the system returns HTTP 429 (Too Many Requests).

Verification

Rate limiting behavior was verified by sending multiple rapid requests to the flight search endpoint. After exceeding the threshold, the system correctly returned HTTP 429 responses, confirming that the gateway enforces request limits effectively.

4. Deployment

The system is deployed on Amazon Web Services (AWS) using an EC2 instance and RDS for database management.

- EC2 Instance: t3.small
- Operating System: Ubuntu 24
- Backend runs on port 8080
- API Gateway runs on port 8081

Both services are packaged as JAR files and executed using nohup, ensuring continuous operation even after terminal disconnection.

The system is publicly accessible via the EC2 public IP address.

5. Project Scope

The system supports the following core functionalities:

- Add Flight
  Allows authorized users to create new flights in the system.

- Add Flight by File
  Enables bulk flight insertion using CSV file upload.
- Query Flight
  Allows users to search for available flights based on date, route, and passenger count. Flights with no available seats are excluded.
- Buy Ticket
  Enables ticket purchasing for available flights and updates seat availability accordingly.
- Check-in
  Allows passengers to check in and assigns seat numbers.
- Query Passenger List
  Provides the list of passengers for a specific flight along with seat information.

6.API Design

API	Parameters	API Response	Description
Add Flight	Flight number, departure date, arrival date, airport from, airport to, duration, capacity	Transaction status	Adds a flight to the airline schedule
Add Flight by File	CSV file with fields: flight number, departure date, arrival date, airport from, airport to, duration, capacity	File processing status	Adds all flights from uploaded file
Query Flight	Departure date, return date, airport from, airport to, number of passengers, trip type (one-way/round trip)	List of available flights (flight number, duration)	Returns available flights. Sold out flights are not listed. The endpoint is rate limited to 3 requests per day via API Gateway. Rate limiting is applied per client IP address.
Buy Ticket	Flight number, date, passenger name(s)	Transaction status, ticket number	Purchases ticket. Decreases capacity. Returns “sold out” if no seats available
Check-in	Flight number, date, passenger name	Transaction status, seat number	Assigns seat to passenger (simple numbering)
Query Flight Passenger List	Flight number, date	List of passengers (with seat numbers)	Returns passenger list of a flight

6.1Endpoint Mapping

API	Endpoint	Method
Login	/api/v1/auth/login	POST
Add Flight	/api/v1/flights	POST
Add Flight by File	/api/v1/flights/upload	POST
Query Flight	/api/v1/flights/search	GET
Buy Ticket	/api/v1/tickets	POST
Check-in	/api/v1/checkin	POST
Query Flight Passenger List	/api/v1/flights/{flightId}/passengers	GET

7. Authentication and Paging

Operation	Authentication	Paging (size of 10)
Add Flight	YES	NO
Add Flight by File	YES	NO
Query Flight	NO	YES
Buy Ticket	YES	NO
Check-in	NO	NO
Query Flight Passenger List	YES	YES

8. Requirements

8.1 Functional Requirements

- The system shall allow authorized users to add new flights to the airline schedule.

- The system shall support uploading a CSV file to add multiple flights at once.

- The system shall allow users to search for available flights based on filters such as date, airport, and number of passengers.

- The system shall not return flights that have no available seats.

- The system shall limit flight search requests to a maximum of 3 per day.

- The system shall allow users to purchase tickets for available flights.

- The system shall decrease the available seat count after a successful ticket purchase.

- The system shall return a "sold out" response if there are no available seats.

- The system shall allow passengers to perform check-in for their flights.

- The system shall assign a seat number to each passenger during check-in.

- The system shall allow retrieval of the passenger list for a specific flight.

8.2 Non-Functional Requirements

- The system shall use RESTful API design principles.

- All APIs shall be versioned (e.g., /api/v1/).

- The system shall use JWT-based authentication for secured endpoints.

- The system shall support paging with a default size of 10 for applicable endpoints.

- The system shall provide API documentation using Swagger/OpenAPI.

- The system shall include an API Gateway for routing and rate limiting.

- The system shall implement rate limiting for specific endpoints.

- The system shall be tested using load testing tools such as k6.

- The system is capable of handling concurrent requests and includes rate limiting to prevent abuse.

9. Assumptions

- Each flight is uniquely identified by a combination of flight number and departure date/time.

- Airport information is represented using standard IATA codes (e.g., ADB, IST).

- Flight duration is stored in minutes.

- The available seat count is initialized as equal to the flight capacity.

- A ticket may include multiple passengers.

- Each passenger is identified by their full name.

- Seat numbers are assigned sequentially during check-in (simple numbering system).

- A passenger can perform check-in only once per flight.

- Only passengers with a valid ticket can perform check-in.

- Query Flight endpoint is publicly accessible, while other critical operations require authentication.

- Rate limiting (3 requests per day) is applied per user or IP address.

- CSV file uploads must follow the predefined format for flight data.

- The system uses MySQL as the relational database management system.

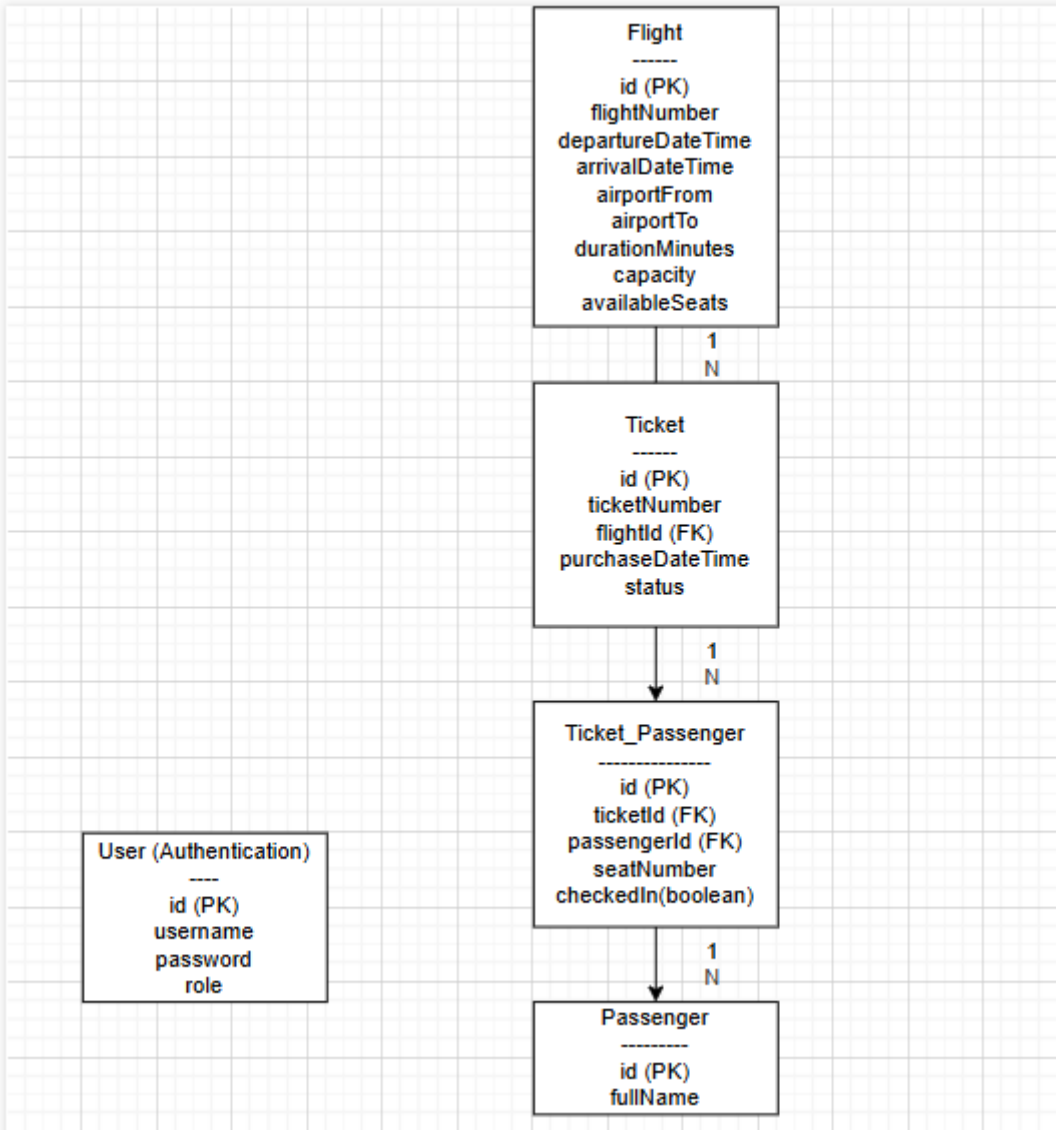
- Rate limiting is enforced at the API Gateway layer to ensure scalability and centralized traffic control.

- The rate limiting mechanism is implemented using a custom in-memory approach for simplicity.

- In a production environment, a distributed rate limiting solution (e.g., Redis) would be preferred.

The Query Flight endpoint currently performs one-way search logic using departure date, route, and passenger count. The tripType and returnDate parameters are accepted for extensibility, but round-trip combination logic is not fully implemented in the current version.

10. Data Model (ER Diagram)



The ER diagram represents the data model of the airline ticketing system. The Flight entity stores flight-related information such as route, schedule, and capacity. Each flight can have multiple tickets associated with it (one-to-many relationship).

The Ticket entity represents ticket purchase operations. Since one ticket may include multiple passengers, the TicketPassenger entity is used to associate passengers with tickets and store additional information such as seat number and check-in status.

The Passenger entity stores passenger information, while the User entity is used for authentication and authorization. This design supports all required operations including flight management, ticket purchase, check-in, and passenger listing.

11.Normal Load

Normal | 20 | 8.26 ms | 2.91 ms | 19.75 req/s | 99.50% | The high failed request rate was expected because the flight search endpoint is protected by gateway rate limiting and returns HTTP 429 after the allowed request threshold is exceeded.

Under normal load (20 virtual users), the system maintained very low response times, with an average response time of 8.26 ms and a p95 of 2.91 ms. The observed high failed request rate does not indicate a system failure; it is mainly caused by expected HTTP 429 responses generated by the API Gateway rate limiting policy on the flight search endpoint. This result shows that the gateway successfully protected the backend service while preserving fast response times for accepted requests.

11.1 Peak Load

Peak | 50 | 593.36 µs | 1.52 ms | 49.93 req/s | 100.00% | The failed request rate mainly reflects expected HTTP 429 responses caused by the API Gateway rate limiting policy.

Under peak load (50 virtual users), the system continued to respond very quickly, with an average response time of 593.36 microseconds and a p95 of 1.52 ms. The failed request rate was expected because the Query Flight endpoint is protected by rate limiting at the API Gateway layer, which returns HTTP 429 when the request threshold is exceeded. This indicates that the gateway effectively controlled traffic while keeping latency low for processed requests.

11.2 Load Testing Results

Scenario	VUs	Avg Response Time	p95	Requests/sec	Error Rate	Notes
Normal	20	8.26 ms	2.91 ms	19.75 req/s	99.50%	High error rate is expected due to rate limiting (HTTP 429)
Peak	50	0.593 ms	1.52 ms	49.93 req/s	100.00%	Rate limiting triggered more frequently under higher load
Stress	100	0.679 ms	2.06 ms	99.86 req/s	100.00%	System remained stable while gateway protected backend

Load testing was conducted on the flight search endpoint through the API Gateway using k6. The system was tested under three scenarios: normal load (20 users), peak load (50 users), and stress load (100 users).

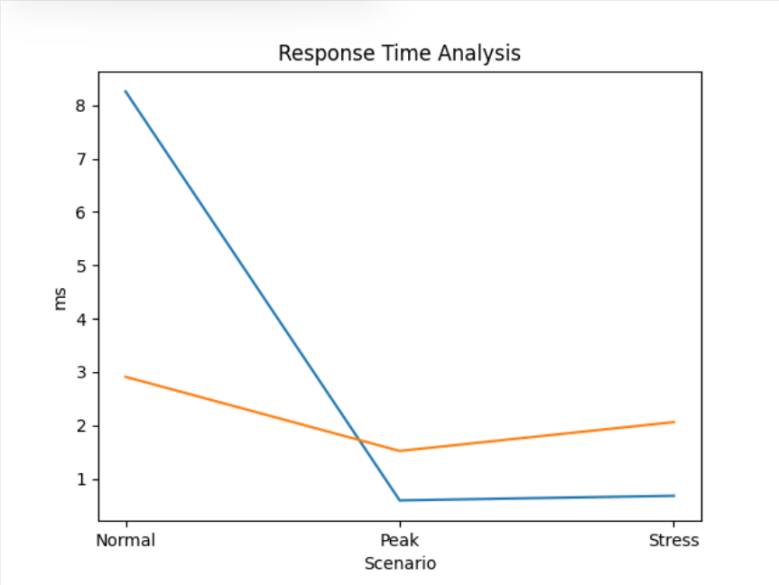
Under all scenarios, the system maintained very low response times, with sub-millisecond performance observed under higher loads. The p95 latency remained below 3 ms across all tests, demonstrating strong performance consistency.

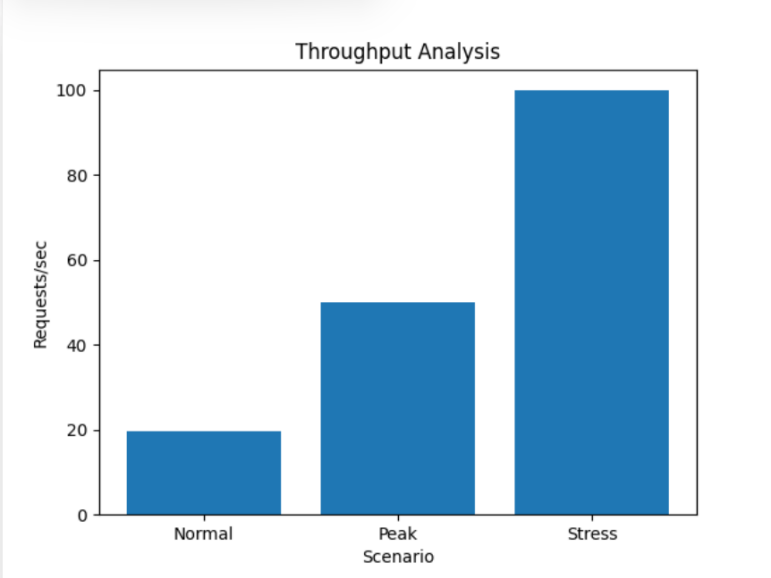
The high error rates observed during testing do not indicate system failure. Instead, they are a direct result of the API Gateway rate limiting policy applied to the flight search endpoint. Requests

exceeding the allowed threshold were intentionally rejected with HTTP 429 (Too Many Requests), confirming that rate limiting is functioning correctly.

Overall, the results show that the system scales efficiently under increased load while maintaining fast response times and protecting backend services from excessive traffic.

11.3 Load Testing Visualization





Throughput increases proportionally with the number of concurrent users. Under stress conditions, the system handled nearly 100 requests per second, demonstrating good scalability. The API Gateway effectively managed traffic while maintaining high throughput.

12.Infrastructure

- EC2 Instance Type: t3.small
- Operating System: Ubuntu 24
- Database: AWS RDS (MySQL)

13.Running Services

- Backend Service (Spring Boot) → Port 8080
- API Gateway (Spring Cloud Gateway) → Port 8081

Both services are packaged as executable JAR files and run as background processes using nohup, ensuring continuous operation even after terminal disconnection.

14.Live System Access

The system is publicly accessible via the following endpoints:

API Gateway (Main Entry Point)

http://13.53.166.181:8081/api/v1/flights

Flight Search Endpoint

http://13.53.166.181:8081/api/v1/flights/search

Swagger API Documentation

http://13.53.166.181:8080/swagger-ui/index.html

All other endpoints can be explored and tested through the Swagger interface.

15.Runtime Configuration

The application uses environment variables for database connection:

```
export DB_URL=jdbc:mysql://<RDS-ENDPOINT>:3306/airline_db
export DB_USERNAME=admin
export DB_PASSWORD=******
```

16.Running the Application (Production Mode)

Build

```
mvn clean package -DskipTests
```

Run Backend

```
nohup java -jar target/airline-api-0.0.1-SNAPSHOT.jar > backend.log 2>&1 &
```

Run Gateway

```
nohup java -jar target/airline-gateway-0.0.1-SNAPSHOT.jar > gateway.log 2>&1 &
```

17.System Status

- Backend service is fully operational
- API Gateway routing is active
- Rate limiting is successfully enforced (HTTP 429)
- Database connectivity is stable
- Swagger documentation is accessible

The system is publicly accessible and ready for demonstration.

Security Note

This project uses Spring Security with basic configuration. In production environments, security configurations should be hardened (e.g., OAuth2, HTTPS, secret management).

18.Conclusion

This project successfully demonstrates the implementation of a scalable and efficient airline backend system using modern technologies.

The use of API Gateway and rate limiting improves system reliability and performance. Load testing results show that the system can handle increasing traffic while maintaining low latency.

Deployment on AWS ensures real-world applicability, making the system suitable for production-like environments.

Overall, the project meets all functional and non-functional requirements and provides a strong foundation for further improvements.